

3. Write a Java program to demonstrate

a) method overloading and method overriding

Method Overloading:

Program :

```
// Class demonstrating method
overloading

class Calculator {

    // Overloaded method for
    integers int add(int a, int b) {
        return a + b;
    }

    // Overloaded method for
    doubles double add(double a,
    double b) {
        return a + b;
    }
}

public class MethodDemo {
    public static void main(String[] args) {

        Calculator calc = new Calculator();

        // Method Overloading
        System.out.println("add(10, 5) = " + calc.add(10, 5));
        System.out.println("add(2.5, 3.5) = " + calc.add(2.5, 3.5));

        /*
        * Uncommenting the following line will cause
        * a compilation error because no matching
        * overloaded method exists.
        */

        // System.out.println(calc.add("10", "5"));
    }
}
```

Method Overriding :

Program :

```
// Parent class

class Parent {
    void display() {
        System.out.println("Parent display method executed");
    }
}

// Child class overriding display()

class Child extends Parent {
    // @Override
    void display() {
        System.out.println("Child display method executed");
    }
}

public class MethodDemo {
    public static void main(String[] args) {

        // Method Overriding
        Child child = new Child();
        child.display();

        // Runtime Polymorphism
        Parent ref = new Child();
        ref.display();

    }
}
```

b) inheritance (single, multilevel) and interfaces (for multiple inheritance)

Single Inheritance :

Program:

// Parent class for Single Inheritance

```
class Person {  
    void showPerson() {  
        System.out.println("Person method executed");  
    }  
}
```

// Child class extending Person class

```
Student extends Person {  
    void showStudent() {  
        System.out.println("Student method executed");  
    }  
}
```

```
public class InheritanceDemo {  
    public static void main(String[] args) {  
        // Single Inheritance  
        System.out.println("Single Inheritance:");  
        Student student = new Student();  
        student.showPerson(); // Parent method  
        student.showStudent(); // Child method  
    }  
}
```

}

Multilevel Inheritance :

Program :

// Multilevel Inheritance

```
class Animal {  
    void eat() {  
        System.out.println("Animal can eat");  
    }  
}
```

```
class Dog extends Animal {
```

```

void bark() {
    System.out.println("Dog can bark");
}
}

class Puppy extends Dog {
void weep() {
    System.out.println("Puppy can weep");
}
}

public class InheritanceDemo {
public static void main(String[] args) {
// Multilevel Inheritance
System.out.println("\nMultilevel Inheritance:");
Puppy puppy = new Puppy();
puppy.eat(); // Animal method
puppy.bark(); // Dog method
puppy.weep(); // Puppy method
}
}

```

b) interfaces (for multiple inheritance):

Program:

```

// Interfaces for Multiple
Inheritance

interface Father {
    void fatherProperty();
}

interface Mother {
    void motherProperty();
}

// Class implementing multiple interfaces

class Child implements Father, Mother {
    @Override
    public void fatherProperty() {
        System.out.println("Father's property method executed");
    }

    @Override
    public void motherProperty() {

```

```
        System.out.println("Mother's property method executed");  
    }  
}
```

```

public class InheritanceDemo {
    public static void main(String[] args) {

        // Multiple Inheritance using Interfaces
        System.out.println("\nMultiple Inheritance using Interfaces:");
        Child child = new Child();
        child.fatherProperty();
        child.motherProperty();
    }
}

```

```

/*

```

```

 * Failure/Edge Cases
 */

```

```

// 1. Interface method not implemented
// Compilation Error Example:
/*

```

```

class Test implements Father {
    // fatherProperty() not implemented
}

```

```

*/

```

```

// 2. Access private parent member directly
// Compilation Error Example:
/*

```

```

class Parent {
    private int age = 25;
}

```

```

class ChildClass extends Parent {
    void display() {
        System.out.println(age); // Error: age has private access
    }
}

```

```

*/

```

